

Mixture-of-Experts

Deep Dive into SGLang — Chapter 21

Yin Yang

Foundation Books Project

Where this chapter sits

This is the chapter where one dense feed-forward step becomes a sparse **route, execute, communicate, combine** path.

- Each token row may travel to a different expert group — but the final output still belongs to the row that requested it.
- SGLang keeps **two ledgers** at once: the *logical* experts the model chose, and the *physical* instances that execute them.
- Sparsity may move work; it may not move request ownership.

Goal: follow one token row through route, dispatch, run, and combine, and say which value changed and which identity must not.

The sparse contract

Route, dispatch, run, combine

Placement, balance, and cost

The sparse contract

One layer, one running trace

Four token rows through a routed layer; top-2 routing turns them into $4 \cdot 2 = 8$ routed assignments:

$$t_0 : \{0, 1\} \quad t_1 : \{0, 2\} \quad t_2 : \{0, 3\} \quad t_3 : \{1, 2\}.$$

- Counting by logical expert: E_0 gets 3, E_1 gets 2, E_2 gets 2, E_3 gets 1.
- In *request* order the trace is four rows wide; in *execution* order it is four expert groups wide.

That conversion — token-major to expert-major and back — is the main source of MoE serving complexity.

The serving contract

Definition (MoE serving contract)

For one routed layer, the runtime must produce the same row-indexed sparse result

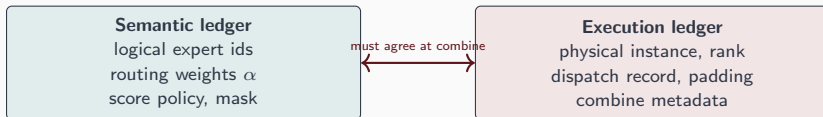
$$y_t = \sum_{j=1}^{k_t} \alpha_{t,j} E_{e_{t,j}}(x_t),$$

even if it *permutes, pads, replicates, or communicates* intermediate token slices.

For t_0 with $\{(E_0, 0.7), (E_1, 0.3)\}$, the final row must equal $0.7 E_0(x_{t_0}) + 0.3 E_1(x_{t_0})$ — even if dispatch renames the physical slots or moves the copies across ranks.

Logical id and routing weight are the invariant; physical placement is free to change.

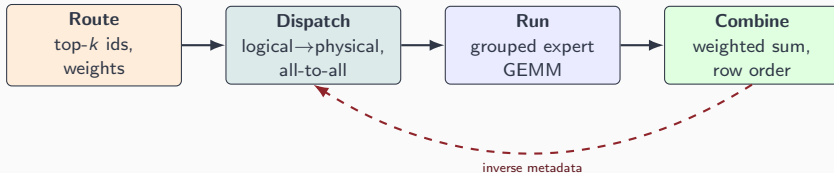
Two ledgers, one row



Correct MoE serving is the agreement between these two ledgers at combine time. A logical expert e names a learned function; a physical instance p names the replica, shard, or rank-local slot that runs it.

Route, dispatch, run, combine

The four stages



- **Route:** the router policy emits ids, weights, and a padding convention — not just k .
- **Dispatch:** map ids through placement, exchange rows across ranks, record inverse state.
- **Run:** many small per-expert MLPs as one grouped kernel.
- **Combine:** reduce with the stored weights back into token order.

Router records: ids are not the whole route

A bare k is insufficient. The executable policy also carries:

- score transform (softmax vs. sigmoid + bias),
- grouping, routed scaling λ , fused shared-expert columns,
- padding convention and top- k output format.

Same logits, different record: for expert logits $[4, 3, 0]$, $k = 2$, a softmax and a sigmoid router may pick the same ids $(0, 1)$ — but softmax weights compete on a simplex while sigmoid weights score independently.

Combine must consume ids and weights under the policy that produced them. Re-ranking a completed record silently changes the route.

Expert-parallel all-to-all



Unlike all-reduce, ranks do not receive the same summed tensor. Each rank gets the subset of routed work for its resident experts; combine uses the stored row ids and weights to reduce results back into the original rows — not the expert-major order.

Grouped GEMM: real rows vs. padding

A dense MLP is one matmul; an MoE runner has **many small per-expert MLPs**, sorted into expert-major blocks and padded to the kernel tile.

Trace loads $(3, 2, 2, 1)$, block size $q = 4$:

$$\text{padded} = q \lceil N_p / q \rceil = (4, 4, 4, 4), \quad 16 \text{ slots, only 8 real.}$$

- `prepare_moe_input`: real counts drive GEMM problem sizes $M = (3, 2, 2, 1)$.
- `moe_align_block_size`: padded slots drive launch shape; a sentinel marks padding.

Padding buys a legal kernel shape — it must never enter the combine sum as model work.

From concept to code: the layer-local execution boundary

Route, dispatch, expert run, and combine all meet in one forward.

```
from python/sglang/srt/layers/moe/fused_moe_triton/layer.py

def forward_impl(self, hidden_states, topk_output):
    # 1) dispatch routed rows to the experts that own them
    dispatch_output = self.dispatcher.dispatch(hidden_states, topk_output)
    # 2) run the selected (quantization-specific) expert core
    combine_input = self.run_moe_core(dispatch_output=dispatch_output)
    # 3) invert the dispatch layout; reduce with routing weights
    final_hidden_states = self.dispatcher.combine(combine_input=combine_input)
    # 4) all-reduce only when configured to reduce MoE TP/EP results
    if self.reduce_results and (self.moe_tp_size > 1 or self.moe_ep_size > 1):
        final_hidden_states = tensor_model_parallel_all_reduce(final_hidden_states)
    return final_hidden_states
```

Dispatch format is a correctness boundary, not a perf detail: once a runner consumes a permuted layout, combine must get matching inverse metadata.

Placement, balance, and cost

EPLB: placement is a mutable runtime object

Sparse activation only helps if routed work is *balanced*. One hot physical expert stalls the whole step:

$$\rho_{\text{imb}} = \frac{\max_p N_p}{\text{mean}_p N_p}, \quad (8, 8, 8, 40) \Rightarrow \rho_{\text{imb}} = \frac{40}{16} = 2.5.$$

EPLB's loop is **count** $\rightarrow$ **plan** $\rightarrow$ **move** $\rightarrow$ **publish**: count logical demand over a window, plan a new replica map, move weights by logical id, publish only at a step boundary.

A **step consumes one placement snapshot** — never old weights mixed with new metadata. EPLB changes where E_0 lives, not that a token chose E_0 .

Cost, correctness, and honest measurement

Invariant: MoE step-alignment

Each token output is reduced from *exactly* the experts selected for that token, under the same routing probabilities used to dispatch it.

Most MoE bugs are **shape-valid but identity-wrong**: a stale placement map, a mixed DeepEP mode, a wrong scale layout.

The cost account keeps terms separate —

$$C_{\text{moe}} \approx C_{\text{route}} + C_{\text{pack}} + \sum_p C_{\text{expert}} + C_{\text{comm}} + C_{\text{combine}},$$

and no throughput number is trusted until a sampled row restores to its original ids and weights.

What to carry forward

1. MoE = one dense step turned into **route, execute, communicate, combine** — ownership survives the detour.
2. Keep the **semantic** and **execution** ledgers separate; they must agree at combine.
3. The **router record** (policy, not just k) is the contract every later stage inherits.
4. **Padding** buys kernel shape; it is never model work.
5. **EPLB** moves placement between steps; each step reads one snapshot.
6. Every stage has a **home in the SGLang source** — that link is the spine of the chapter.

Next: Section 1 states the serving contract that turns tensor row order into request ownership — and names the two ledgers that must agree.